Learning algorithms

Meeting 2. Neural machines

Course meeting 22

In Meeting 1, we separated learning a model from using it, and asked whether its predictions work on examples it has not studied. We now open up one model family: a neural network. The same questions remain. What changes during fitting? What stays fixed during prediction? Which observations should guide our choices?

Recall the single neuron. It takes inputs, computes a weighted sum, and applies an activation function. What changes when we connect many neurons? The computation gets larger, but the basic learning idea stays with us: predict, measure the loss, find gradients, and update the weights. The question is whether those updates teach the model a useful pattern or only the details of the examples it has seen.

The main points

1. **A network is a computation with adjustable numbers.**
   Weights and biases determine how the inputs become a prediction. Fitting learns these numbers from examples.
2. **Computing a prediction, finding a direction, and making a change are separate steps.**
   The forward computation produces a prediction. Backpropagation computes sensitivities. The optimizer uses them to update the model.
3. **Count examples, updates, and passes separately.**
   Several training examples can contribute to one update. Predicting several new examples does not itself train the model.
4. **Some numbers are learned; other settings are chosen.**
   Use validation to compare network configurations under a stated measure. Keep the test set for the final assessment.
5. **Perfect recall need not mean useful prediction.**
   A large network can fit random training labels while remaining near chance on fresh labels.
6. **The last training state need not be the best choice.**
   Compare saved states on validation examples. Continuing to reduce training error can make new predictions worse.

1. A task that needs a curved boundary

Each observation in our first example has two numerical **features**, the input measurements, and a **label**, the observed class, either 0 or 1. The points form two interleaving crescent shapes. Can one straight line separate them? Look at the picture before choosing a model. A straight boundary cannot follow the curved arrangement.

There are 240 training observations, used to fit the model, and 120 validation observations, used to compare choices. Another 120 observations stay reserved for the final test. The figures and comparisons below do not use test answers.

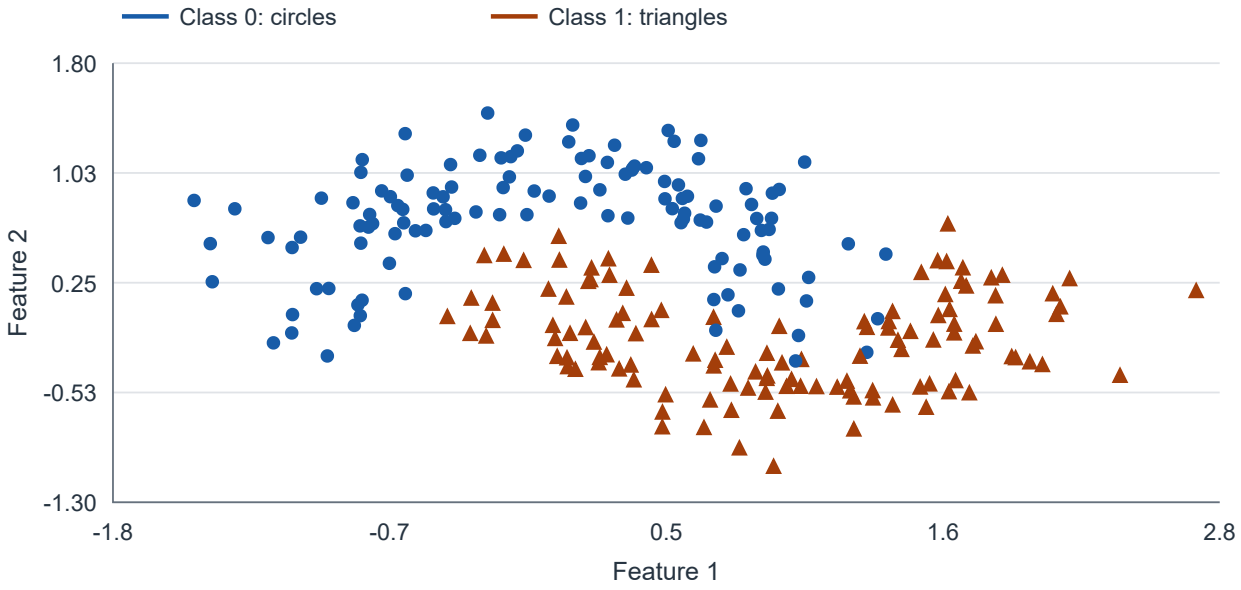

Figure description: Scatter plot of 240 training observations. The horizontal axis is feature 1, from minus 1.8 to 2.8; the vertical axis is feature 2, from minus 1.3 to 1.8. Class 0 uses blue circles in an upper crescent; class 1 uses rust-colored triangles in a lower crescent. The crescents interleave and overlap near their ends, so a straight separator misses their curved arrangement.

The 240 training observations. Circles mark class 0; triangles mark class 1. A useful predictor needs to follow the curved arrangement while allowing for overlap and noise.

A **decision boundary** is where the predicted class changes. Fitting a flexible boundary may help, but a training picture alone cannot tell us how well that boundary predicts new labels. We will return to its measured validation performance after explaining the computation and its training process.

2. From inputs to a prediction

Weights, biases, and hidden layers

A neuron multiplies each input by a **weight**, adds the results, and adds a **bias**, an adjustable offset. It then applies an **activation function** to that sum. Weights and biases are the model's learned **parameters**. The activation function specifies how each neuron transforms its sum.

$$z = w_1 x_1 + w_2 x_2 + b, \qquad h = \phi(z)\,.$$

In words: The weighted sum z equals weight one times input one, plus weight two times input two, plus bias b. The neuron output h is the activation function phi applied to z.

Here the inputs are `x1` and `x2`, the weights are `w1` and `w2`, the bias is `b`, and the activation function is denoted by the Greek letter phi. The output of this neuron is `h`. A **hidden layer** is a collection of intermediate neurons: its values become inputs to the next layer rather than being the final answer.

Two activation functions

The **sigmoid** turns any real input into a value between zero and one. The **rectified linear unit (ReLU)** returns zero for a negative input and leaves a positive input unchanged. Both return fixed functions of their inputs; training adjusts the surrounding weights and biases.

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \mathrm{ReLU}\,(z) = \max\,(0, z)\,.$$

In words: Sigmoid of z equals one divided by the quantity one plus e raised to minus z. ReLU of z is the larger of zero and z.

For sigmoid, `e` is the base of the natural exponential. Sigmoid at zero is one half. ReLU at minus 3 is zero, and ReLU at 2 is 2; its output is not restricted to probabilities.

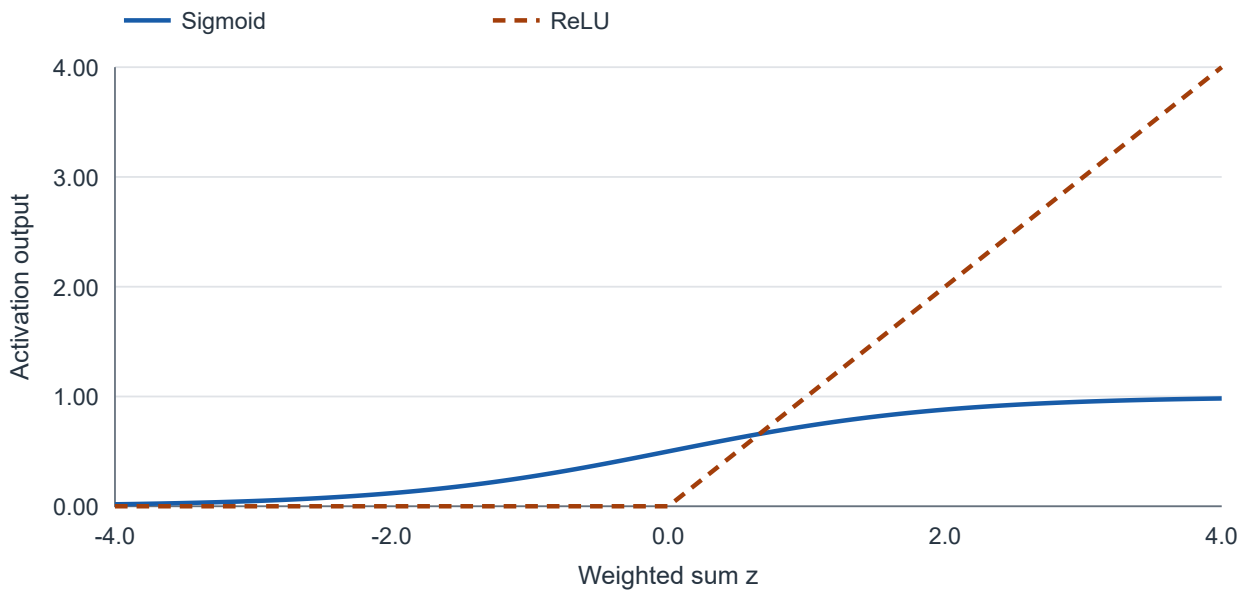

Figure description: Line graph with weighted sum z on the horizontal axis from minus 4 to 4 and activation output on the vertical axis from 0 to 4. The solid blue sigmoid rises smoothly from near zero to near one, passing through one half at zero. The dashed rust-colored ReLU is zero for nonpositive inputs, then follows the input: at z equal to 2 its output is 2. Thus ReLU output is not a probability.

Two ways to transform a weighted sum. The sigmoid curve is smooth and stays between zero and one. ReLU has a corner at zero and rises as a straight line for positive inputs.

These nonlinear transformations matter. Composing weighted sums and biases without any nonlinear hidden activation would still give an affine rule: a weighted sum plus an offset. In our binary classifier, that would still yield a straight decision boundary.

A network small enough to follow

The large network repeats the single-neuron calculation at many places. To see what is happening inside `fit`, use a network small enough that we can follow every value. The hidden neurons compute their outputs, and those outputs become the next neuron's inputs.

Use the input pair (1, 2). Our small network has two sigmoid hidden neurons and one sigmoid output. Its four input-to-hidden weights are 0.5, 0, minus 0.5, and 0. The two hidden-to-output weights are 1 and minus 1. All three biases start at zero. There are nine adjustable parameters in total.

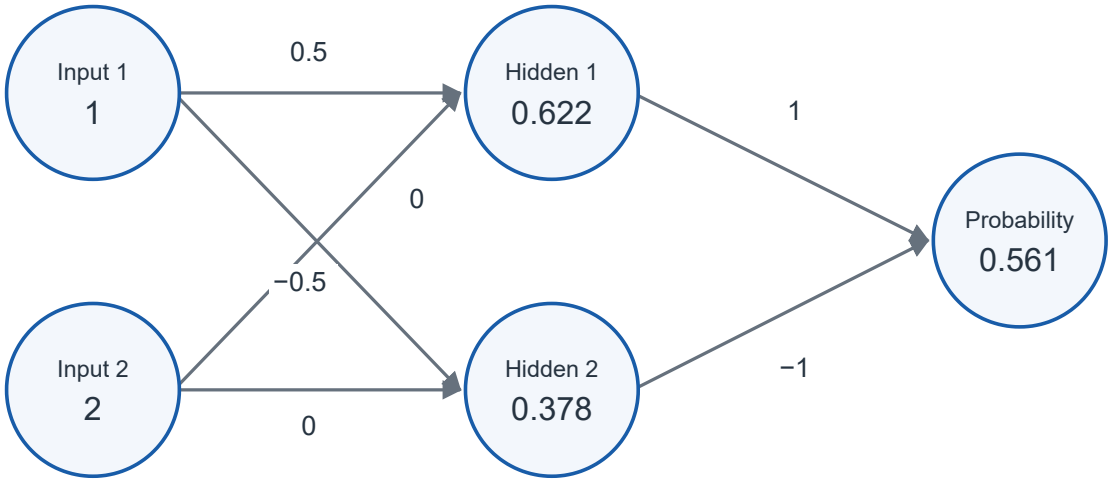

Figure description: Network diagram flowing left to right. Inputs 1 and 2 have values 1 and 2. Hidden neuron 1 receives weights 0.5 and 0; its weighted sum is 0.5 and its sigmoid value is about 0.622. Hidden neuron 2 receives weights minus 0.5 and 0; its sigmoid value is about 0.378. Output weights are 1 from the first hidden neuron and minus 1 from the second. The output score is about 0.245; its sigmoid gives probability 0.561 for class 1. All three biases start at zero. There are nine trainable parameters.

The supplied starting network. Arrows carry the labelled weights. Each hidden value is computed before it is used at the output; zero-weight connections are still part of the model and can change during training.

A **forward computation** evaluates the network from inputs toward its output, using its current parameters. The first hidden sum is 0.5 times 1 plus 0 times 2, which is 0.5. The second is minus 0.5. Applying sigmoid gives hidden values approximately 0.622 and 0.378.

$$h_1 = \sigma(0.5) \approx 0.622, \qquad h_2 = \sigma(-0.5) \approx 0.378\,.$$

In words: The first hidden value is sigmoid of zero point five, approximately zero point six two two. The second is sigmoid of minus zero point five, approximately zero point three seven eight.

The output first forms a **score**, also called a **logit**, from the hidden values. Applying sigmoid to this score gives the predicted probability `p` of class 1.

$$s = h_1 - h_2 \approx 0.245, \qquad p = \sigma(s) \approx 0.561\,.$$

In words: The output score s is the first hidden value minus the second, approximately zero point two four five. The predicted probability p is sigmoid of s, approximately zero point five six one.

With a decision threshold of one half, probabilities at least one half become class 1 and lower probabilities become class 0. Thus this row is predicted as class 1. The threshold produces a label; it does not change the weights or train the network.

3. Forward, backward, update

Give the prediction a loss

A **loss** assigns a numerical penalty to a prediction compared with its observed answer. Fitting tries to reduce that penalty. For binary labels we use **log loss**, also called binary cross-entropy or negative log-likelihood for the observed label. If the correct label is 1, the loss is minus the natural logarithm of the predicted probability of class 1; if it is 0, use the probability of class 0 instead.

$$L = -\left[y\log(p) + (1-y)\log(1-p)\right].$$

In words: Binary log loss L is minus the sum of y times the natural logarithm of p, and one minus y times the natural logarithm of one minus p. Here y is the observed zero-or-one label and p is the predicted probability of class one.

The observed label `y` for the tiny example is 1, so only the first term contributes. At probability 0.561 the loss is about 0.578. Giving the correct class more probability lowers this loss, even if the predicted class label was already correct.

$$y = 1: \qquad L = -\log(p) \approx 0.578\,.$$

In words: When the observed label y equals one, the loss is minus the natural logarithm of p, approximately zero point five seven eight for this prediction.

Find how the parameters affect the loss

A **gradient** collects derivatives of the loss with respect to the parameters. Each entry measures the local sensitivity to changing one parameter while holding the others fixed. A negative entry means a sufficiently small increase in that parameter lowers the loss at the current state. A positive entry suggests moving in the opposite direction.

What travels backward? In the forward computation, we pass activation values toward the output. **Backpropagation** works backward from the loss, passing information about derivatives. An early weight affects a hidden value; that affects the output score; that affects the loss. The chain rule combines those local effects to compute the sensitivities. We are finding how the loss would change. At this stage, the weights themselves have not changed.

Follow one backward path

For sigmoid followed by binary log loss, the derivative with respect to the output score is `p - y`, here about minus 0.439. The first hidden neuron reaches the score through a weight of 1. Its sigmoid derivative is its value times one minus that value. Finally, the first input is 1. Multiplying these effects gives the derivative for its first weight, `w11`:

$$\frac{\partial L}{\partial w_{11}} = (p - y) \cdot 1 \cdot h_1(1 - h_1) \cdot x_1 \approx -0.103184\,.$$

In words: The derivative of loss with respect to the first input-to-hidden weight equals p minus y, times the output connection weight one, times h one times one minus h one, times the first input x one. Its value is approximately minus zero point one zero three one eight four.

You do not need to derive every entry to interpret this result: a small increase in this weight should reduce the current loss. The supplied computation checks all nine derivatives against small numerical perturbations.

Use the gradient to make an update

An **optimizer** is the rule that updates the parameters using the gradients. Ordinary **gradient descent** subtracts the gradient multiplied by a positive **learning rate**. The learning rate sets the scale of the move; its Greek symbol is eta.

$$w_{\text{new}} = w - \eta \frac{\partial L}{\partial w}\,.$$

In words: The new weight equals the old weight minus the learning rate eta times the derivative of loss with respect to that weight.

For the first weight, the old value is 0.5, the derivative is about minus 0.103184, and the learning rate is 0.5:

$$w_{\text{new}} = 0.5 - 0.5(-0.103184) \approx 0.551592\,.$$

In words: The new weight is zero point five minus zero point five times minus zero point one zero three one eight four, approximately zero point five five one five nine two. The negative derivative makes the weight increase.

Subtracting a negative quantity increases this weight. The example first computes the gradients at the old parameter values, then updates all nine parameters. Working backward through layers describes the order of the derivative calculation; it does not mean changing one layer before finding the gradients for the next. A new forward computation then gives probability 0.673 and loss 0.395. These are the results of updating the whole network, not just the highlighted weight.

Figure description: Two curves show the tiny network evaluated after zero through four complete parameter updates. The horizontal axis counts updates and the vertical axis runs from zero to one. The solid blue probability curve with circles rises from 0.560925 to 0.827310. The dashed rust-colored loss curve with crosses falls from 0.578167 to 0.189576. Both use the same input and observed label 1. This is a training computation on one row, not evidence of generalization.

Four gradient-descent updates on the same supplied row, with learning rate 0.5. The probability of the correct class rises while the loss falls. This small computation explains a training step; it does not establish performance on new examples.

The numerical trace

Recompute the prediction after each completed update

Updates completed	Probability of class 1	Log loss
0	0.560925	0.578167
1	0.673412	0.395398
2	0.745520	0.293674
3	0.793626	0.231143
4	0.827310	0.189576

Now do it again. The next forward computation uses the updated weights, so it can produce a different prediction and loss. Then we compute new gradients and make another update. That repeated cycle is the high-level structure hidden inside `fit`.

What stays fixed when we predict?

Inference means using a fitted model to make predictions. Ordinary inference needs the forward computation, but no backward computation or parameter update. Training also contains forward computations: it needs predictions before it can measure a loss.

Do the activations stay fixed too? No. An activation is a value computed for an input. Different inputs can produce different activations and predictions while using exactly the same weights. In the networks here, fitting changes weights and biases; it does not add neurons, rewire the layers, or change the chosen activation function.

This also helps with the chatbot question from class. A model with fixed weights can respond differently when given more conversation as input. Changing the context is different from updating the parameters. Systems can also be designed to keep training on new data; incremental learning is an existing approach. The distinction here is between ordinary prediction and a training update, not between what humans and machines can ever learn.

4. A local direction is not a map

Imagine walking downhill in fog. You can feel the slope near your feet, but you cannot see every valley. In our model, position means the parameter values and height means training loss. A small move opposite the gradient points downhill locally. It does not reveal the lowest point in the entire landscape, and a large move can overshoot a valley and increase the loss.

A **local minimum** has no smaller loss in a sufficiently small surrounding neighborhood. A **global minimum** is lowest over the whole allowed region. Give the same model two different starting sets of weights. Must training end at the same place? No: different starts can lead to different local minima. That is one reason the initialization matters. At a stationary point the gradient is zero, but that alone does not prove the point is a minimum.

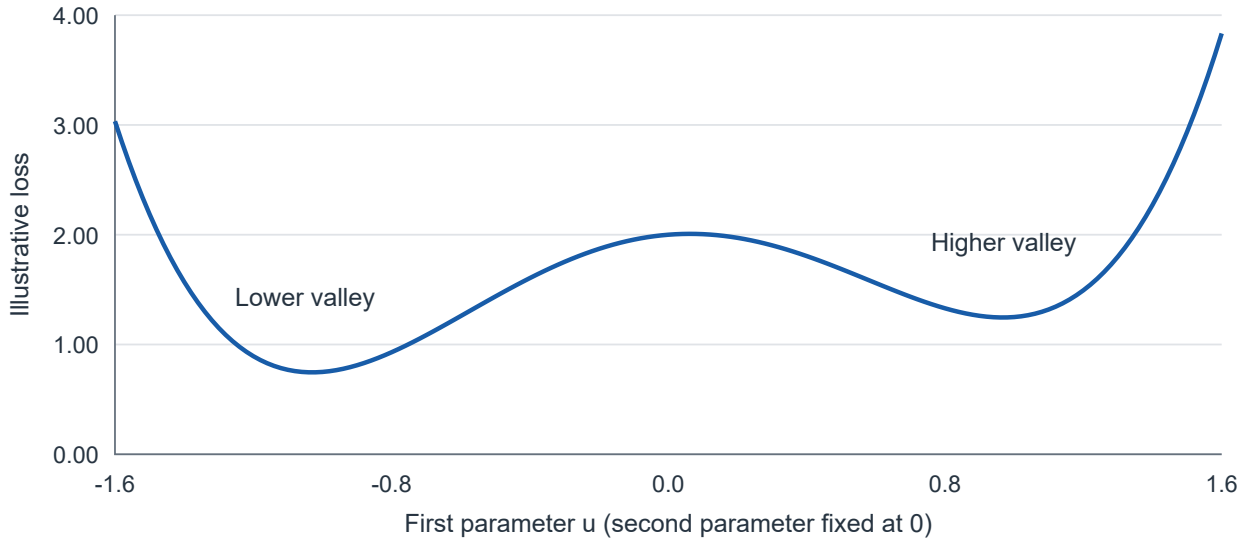

Figure description: A one-dimensional slice of the lecture objective: the square of u squared minus one, plus one quarter of u, plus 1, with the second parameter fixed at zero. The horizontal axis is the first parameter u, from minus 1.6 to 1.6. The vertical axis is illustrative loss, from 0 to 4. There are two valleys near u equal to minus 1 and plus 1; the left valley is lower. A hill near zero separates them. A local minimum in the right valley need not be the best location overall. This is not a measured neural-network loss curve.

An illustrative slice through the lecture's two-parameter landscape, holding the second parameter at zero. The left valley is lower than the right. This is a teaching objective, not the measured loss surface of one of our networks.

The step-size discussion and the validation discussion answer different questions. An optimizer seeks lower training loss. Validation asks whether the resulting predictions work on held-out examples. Even reaching the lowest training loss would not settle that second question.

5. Batches, epochs, and inference

A **mini-batch** is a group of training rows processed together. In our setup, we compute a loss for each row, average those losses, compute the gradient of that average, and make one optimizer update. The next batch uses the newly updated parameters. Keep the counting rule in mind: **one update per batch** in this training loop.

An **epoch** is one complete pass through the training set. It is not the same as one update. The number of updates depends on the batch size. When we keep the final smaller batch and make one update per batch:

Counting one epoch in the lecture examples

Training rows	Batch size	Batches in one pass	Updates
8	4	$4 + 4$ rows	2
240	32	Seven groups of 32, then 16	8
240	240	One group of 240	1

Why see the same rows again? The weights have changed since the previous pass, so we compute new predictions and gradients. Several epochs revisit the training observations, usually in a newly shuffled order. A **training budget** specifies how much training to allow, such as 400 epochs. A **checkpoint** is a saved model state at a particular stage of training. We can compare saved checkpoints without assuming the last one is best.

Predicting a group of rows

Inference can also process a batch, but this only groups the predictions. The same learned weights are used for every row. In scikit-learn, `X_valid` is the validation input table; each row has two features. The slice `[:1]` keeps a table with one row, rather than dropping its row dimension.

```
one = model.predict(X_valid[:1])
many = model.predict(X_valid[:8])
```

The input shapes are (1, 2) and (8, 2): rows first, features second. These calls produce one and eight predictions respectively. Neither call makes a training update. Batch size during inference does not determine how many epochs the model has trained.

> **Reflection: what changes?**
>
> What changes during `fit`? During `predict`, do weights change? Do activations change? Explain where backpropagation and the optimizer enter the training cycle.

6. What we learn and what we choose

The trained weights and biases are learned parameters. **Hyperparameters** are settings chosen for the learning process or model family: the number of layers, the number of neurons per layer, the hidden activation, learning rate, batch size, and training budget. A hidden layer's **width** is its number of neurons. Ask which choices we make about the learning process, and which values the fitting procedure learns. Some settings, such as the learning rate, can also follow a chosen schedule during training.

Regularization modifies fitting to discourage certain solutions. The comparison here uses a small L2 penalty, which penalizes large squared weights; its strength is another hyperparameter. We also use **Adam**, an optimizer that uses running estimates from gradients to adjust its updates. Adam follows the same forward/backward/update separation, but its update is not identical to the elementary gradient-descent calculation above.

The classification networks have two input features, two hidden layers of 16 neurons, and one output. Each neuron has one bias, so the parameter count is:

$$(2 + 1) \cdot 16 + (16 + 1) \cdot 16 + (16 + 1) \cdot 1 = 337\,.$$

In words: The first hidden layer has two input weights and one bias for each of sixteen neurons, giving forty-eight parameters. The next has sixteen weights and one bias for each of sixteen neurons, giving two hundred seventy-two. The output has sixteen weights and one bias, giving seventeen. The total is three hundred thirty-seven parameters.

Changing the activation keeps this count at 337. Changing the widths generally changes it. The value 0.551592 in the tiny-network example is a learned weight; the choice of 16 hidden neurons is a hyperparameter.

Prepare the inputs, then fit

Standardization subtracts a feature's training mean and divides by its training standard deviation. Fit this transformation on training rows only, then use those same stored values for validation or new inputs. Here `X_train` contains the training inputs and `y_train` their labels; `X_valid` contains validation inputs.

The classifier through fit and predict

This excerpt assumes the inputs have already been standardized using training statistics. In scikit-learn, `MLPClassifier` means a multilayer perceptron classifier, the feed-forward network used here. `activation="logistic"` selects sigmoid hidden units; change it to `"relu"` for the other candidate. The remaining settings are kept fixed. We disable automatic stopping based on validation, introduced in Section 8, to inspect the stated training budgets.

```
from sklearn.neural_network import MLPClassifier

model = MLPClassifier(
    hidden_layer_sizes=(16, 16), activation="relu",
    solver="adam", learning_rate_init=0.01,
    batch_size=32, alpha=0.0001, random_state=22,
    max_iter=400, tol=0.0, n_iter_no_change=401,
    early_stopping=False
)
model.fit(X_train, y_train)
labels = model.predict(X_valid)
probabilities = model.predict_proba(X_valid)[:, 1]
```

`alpha` sets the L2-penalty strength. `random_state` fixes the random seed. With Adam, `max_iter` sets the epoch budget, not the number of individual updates. These tolerance settings let this comparison reach the chosen budget; reaching it is not a claim that optimization has converged. The `early_stopping` option is disabled here; Section 8 explains stopping-point selection. The last line selects the probability for class 1. API reference.

Where do the colors in the picture come from?

Take a grid of locations covering the picture and ask the fitted model to predict at each one. Shade each location according to its predicted class. Where the prediction switches, we see a decision boundary. Drawing this picture is inference: it does not supply new training labels or update the weights. The validation points are then placed over the picture so that we can compare predictions with their observed classes.

Which model would you keep?

Accuracy is the fraction of class labels predicted correctly. Mean log loss also considers the probabilities, so it can distinguish two models that predict the same labels with different confidence. An evaluation **metric** is the measure used for comparison. Choose it before selecting a winner.

The two candidates use the same data split, standardization, layer widths, Adam settings, batch size, L2 penalty, random seed, and training budget. Their hidden activation differs. A fixed seed supports repeatability, but it need not make initialization identical across activation types.

Figure description: Decision-region plot for the two-feature moons task. The horizontal axis is feature 1 and the vertical axis is feature 2, on the same ranges as the initial scatter plot. Plain pale regions predict class 0 and striped regions predict class 1. Hollow circles are true class 0 and hollow triangles true class 1; all 120 plotted points are validation observations. A point in the opposite predicted region is a mistake. ReLU hidden activation, 400 training epochs. Training accuracy 97.50%; validation accuracy 97.50%; validation log loss 0.04773. This is a recorded fit, and the validation labels did not update its weights.

Measured predictions on the moons example. The background gives the predicted class: plain for 0 and striped for 1. Hollow circles and triangles are the 120 validation observations, with shapes identifying their actual classes.

Same task and 400-epoch budget

Hidden activation	Training accuracy	Validation accuracy	Validation log loss
Sigmoid	97.92%	96.67%	0.06218
ReLU	97.50%	97.50%	0.04773

Look first at training accuracy: sigmoid is slightly ahead. Is that enough to choose it? At 400 epochs, ReLU has the better validation accuracy and lower validation log loss in this run. That is not a universal ranking. Across the saved budgets, validation accuracy reaches 98.33% for both activations: sigmoid at 200 epochs, and ReLU at 50, 100, and 200. Lower log loss and higher accuracy need not select the same checkpoint.

The controls show actual fits at the displayed budgets, each started from the fixed seed. They do not interpolate an invented training path. Validation answers two selection questions: which configuration should we keep, and which saved version of that configuration should we keep? The last version produced by training need not be the best one. Repeatedly trying settings can itself adapt our decisions to the validation sample; the reserved test set remains separate.

7. What if the labels are coin flips?

Getting familiar practice questions right does not by itself show that we can solve new questions. The model faces the same distinction. **Generalization** means that what was learned supports predictions on new examples from the intended setting. To separate this from memorization, give 80 training points labels produced by independent fair coin flips. Their coordinates contain no population information about those labels. A separate fresh sample has 400 independently generated points and labels.

Capacity refers to the range of patterns a model can represent. Compare one hidden layer of four ReLU neurons with two hidden layers of 64. Both use Adam, learning rate 0.01, a batch containing all 80 training rows, no L2 penalty, and 2,000 epochs.

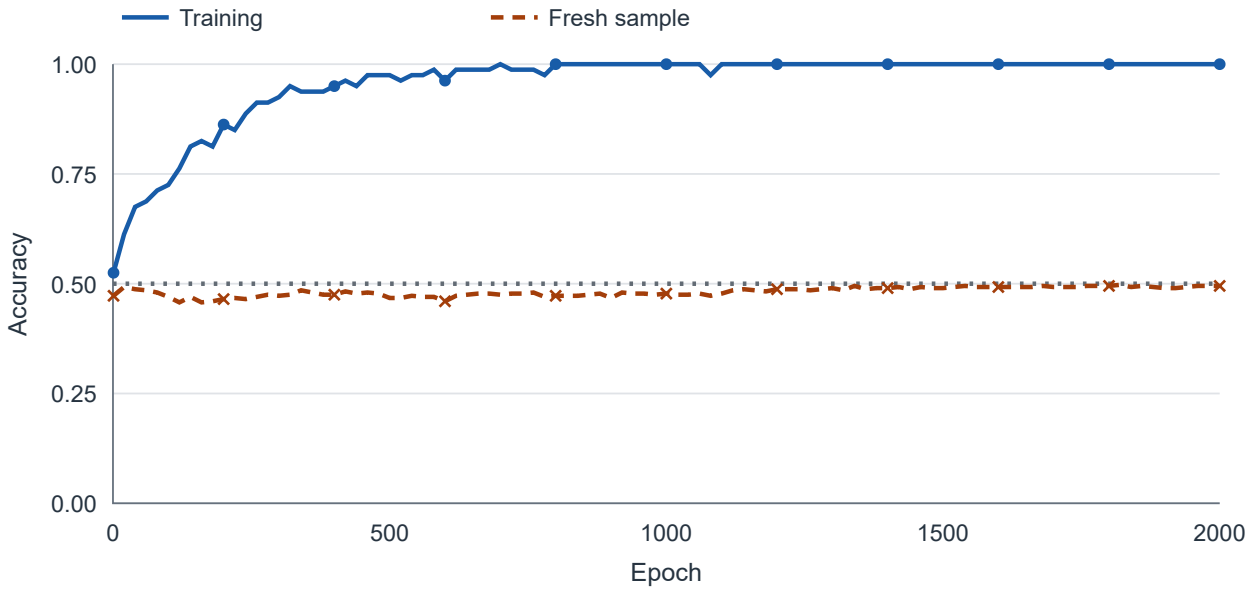

Figure description: Accuracy against training epoch for independent random labels, with epochs zero to 2000 horizontally and accuracy zero to one vertically. The solid blue training curve has circle markers; the dashed rust-colored fresh-sample curve has cross markers. A gray dotted line marks chance accuracy one half. Two hidden layers of 64: 4,417 parameters. At epoch 2000, training accuracy 100.00% and fresh-sample accuracy 49.50%. The training curve reaches one while the fresh-sample curve stays near one half, showing memorization without useful prediction of fresh labels. The fresh sample has 400 observations and the training sample has 80.

Accuracy while fitting independent random labels. The solid line with circles is training accuracy; the dashed line with crosses is accuracy on the fresh sample. The dotted reference is 50%.

Random-label results after 2,000 epochs

Hidden layers	Parameters	Training accuracy	Fresh accuracy
4	17	58.75%	47.75%
64, 64	4,417	100.00%	49.50%

Before revealing the fresh labels, predict the score. There was no relationship to learn between location and an independent coin flip. The larger network gets every training label right but only 49.50% of the fresh labels right. It has learned a computation that reproduces this training sample. It has not discovered a predictive relationship between coordinates and independently generated future labels. Expected fresh accuracy is 50%; a finite sample need not give exactly 50%.

The smaller network's difficulty fitting these labels is not, by itself, a test that proves a dataset has no pattern. Here we know there is no predictive relationship because of how we generated the labels. Nor does the example say that memorization and useful learning cannot coexist. With structured data, a network may learn the broad pattern and also fit sample-specific details.

Reflection: what did the model learn?

Explain the training/fresh-sample gap without blaming the labels. What would training accuracy alone have led you to believe? Why would more fitting not create a population relationship that is absent from the data-generating process?

8. Structure, noise, and a stopping point

What changes when there is a pattern? Now give the model something real to learn. The numerical target is a smooth sine relationship plus independent random variation, or **noise**:

$$y = \sin(3x) + \text{noise}.$$

In words: The observed target y equals sine of three times the input x, plus independent random noise.

There are 30 training observations and 200 validation observations. The added noise has a normal distribution with mean zero and standard deviation 0.35. The model sees noisy targets, not the underlying sine curve. The curve is shown only because we constructed this synthetic example.

We evaluate with **mean squared error (MSE)**: subtract each prediction from its observed target, square each difference, and average. MSE is an error measure and can also serve as a regression loss. Lower is better; this differs from accuracy, where higher is better. A small network has eight hidden neurons; a larger one has two hidden layers of 64. Both use ReLU and the same training settings, with all 30 rows in each batch.

Overfitting occurs when fitting training-specific details harms prediction on new examples. To see it here, compare early and late checkpoints of the same large network. The earliest checkpoint with the smallest validation MSE among those inspected is at epoch 75.

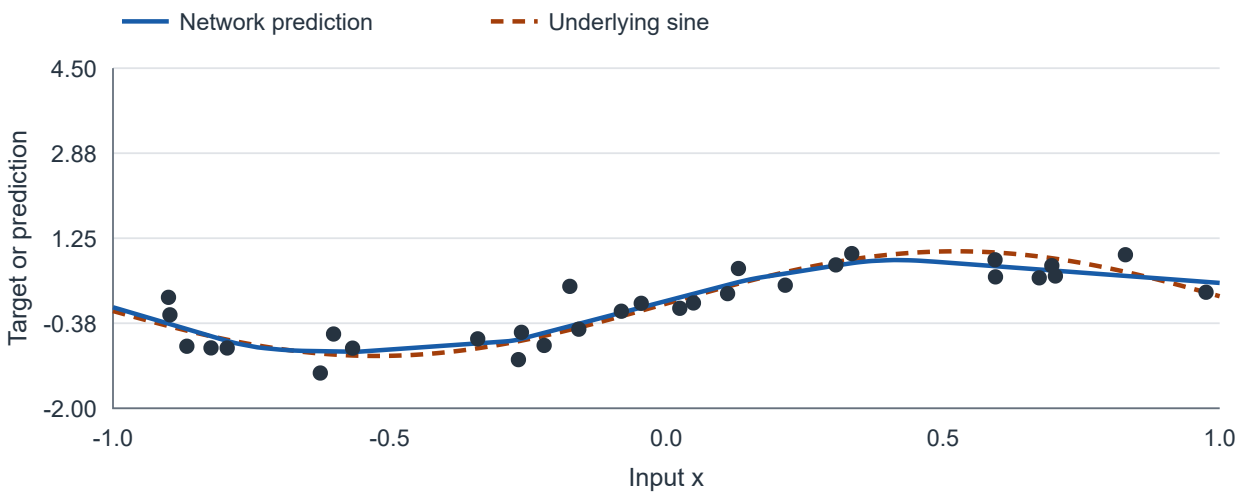

Figure description: Input x runs from minus one to one horizontally; target or prediction runs from minus two to 4.5 vertically. Circles are 30 noisy training observations. The solid blue curve is the prediction; the dashed rust-colored curve is the known sine signal, not supplied to the model. Large network, epoch 75: training MSE 0.07094; validation MSE 0.14313. The early fit follows the broad pattern. Validation MSE uses all 200 validation rows, including edge inputs beyond the training range.

Training points are circles. The solid fitted curve uses only training observations. The dashed sine curve is the underlying signal, shown for interpretation; it is not supplied to the model or used to choose a checkpoint.

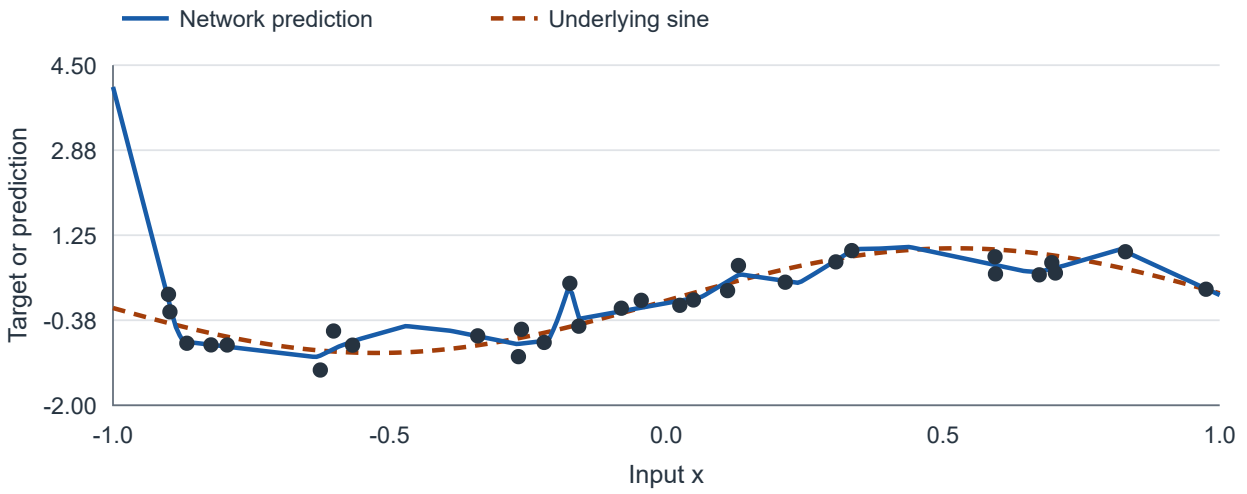

Figure description: Input x runs from minus one to one horizontally; target or prediction runs from minus two to 4.5 vertically. Circles are 30 noisy training observations. The solid blue curve is the prediction; the dashed rust-colored curve is the known sine signal, not supplied to the model. Large network, epoch 2500: training MSE 0.01888; validation MSE 0.25658. The late fit follows more small fluctuations and rises above 4 at the left edge. Validation MSE uses all 200 validation rows, including edge inputs beyond the training range.

The same large network at the final checkpoint. Compare this fit with the earlier one above using the same axes.

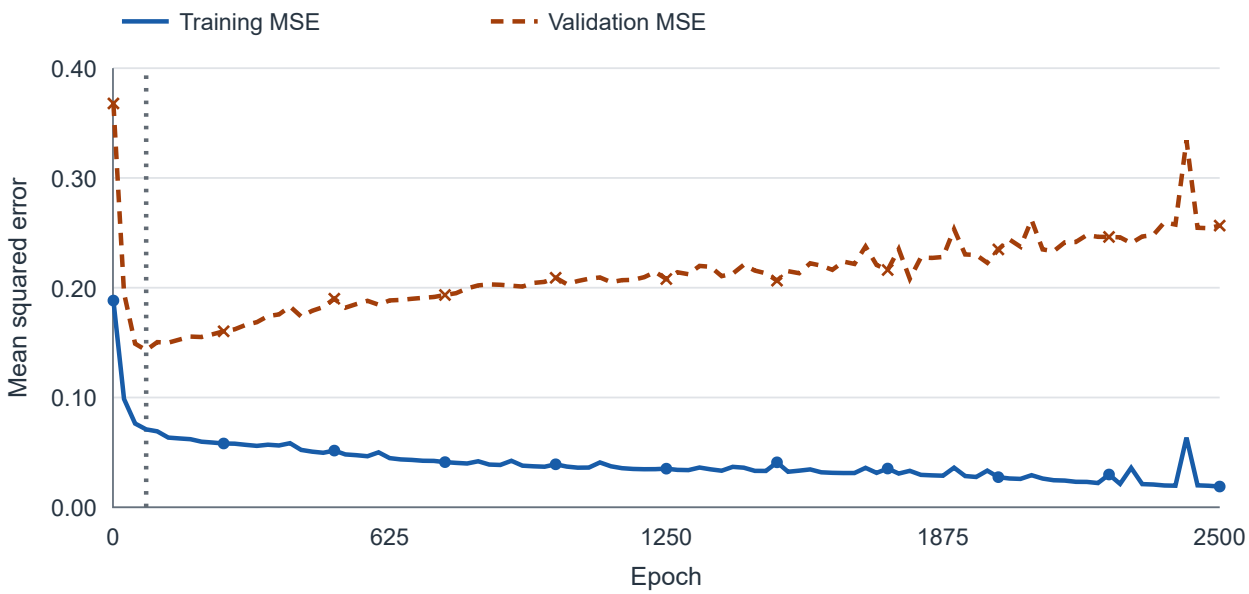

Figure description: Line graph of the large regression network's mean squared errors. The horizontal axis is epoch count, from zero to 2500, and the vertical axis is MSE, from zero to 0.4. The solid blue training curve with circle markers generally falls, with occasional spikes. The dashed rust-colored validation curve with cross markers first falls, reaches its best checked value 0.14313 at epoch 75, and later rises to 0.25658. Training MSE at those two states falls from 0.07094 to 0.01888. A vertical dotted line marks epoch 75. This comparison of the same model at two times demonstrates overfitting in this run.

The large network's recorded training and validation MSE. The horizontal axis is epoch count. The marked validation minimum is among the saved measurements, not a claim about every intermediate update.

Compare the same large network at two times

Checkpoint	Training MSE	Validation MSE
Epoch 75	0.07094	0.14313
Epoch 2500	0.01888	0.25658

The large model's training MSE falls from 0.07094 to 0.01888, while validation MSE rises from 0.14313 to 0.25658. Keeping the earlier saved state illustrates **early stopping**: use validation to choose when to stop retaining further training changes. Here we select manually among recorded checkpoints; we did not enable the library's automatic early-stopping option.

Do not turn this example into “larger is always worse.” The final small model has validation MSE 0.27340, which is slightly worse than the final large model's 0.25658. The clear overfitting comparison is the early versus late large model. The validation sample also includes edge inputs outside the observed training range; its score includes those harder cases.

A checkpoint selected with validation still needs a separate final assessment. Choosing the best validation score and reporting that same score as independent test evidence would repeat the mistake discussed in Meeting 1.

9. Follow-up: the training loop in PyTorch

This section extends the recorded lesson, which ends after the overfitting reflection. Use it with the companion notebook to connect the forward/backward/update cycle to explicit code.

A PyTorch **tensor** is an array of numbers used in the computation. A **module** is a component such as a layer or a complete network. A plain module does not supply scikit-learn's universal `fit` / `predict` interface, so we write the loop explicitly.

`nn.Linear` computes weighted sums plus biases, and `nn.Sequential` connects the listed components in order. Our final layer returns one logit per row. `BCEWithLogitsLoss` combines sigmoid and binary log loss internally for numerical stability; do not apply another sigmoid before passing scores to it. Loss reference.

Construct the network and its training batches

The arrays `X_train` and `X_valid` use the same training-fitted standardization described above. `y_train` contains zeros and ones. `TensorDataset` pairs each row with its label, and `DataLoader` supplies shuffled mini-batches. Targets are floating-point values with one column, matching the output shape.

```
import torch
from torch import nn
from torch.utils.data import TensorDataset, DataLoader

torch.manual_seed(22)
X_tensor = torch.tensor(X_train, dtype=torch.float32)
y_tensor = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)
loader = DataLoader(TensorDataset(X_tensor, y_tensor),
                    batch_size=32, shuffle=True)

model = nn.Sequential(
    nn.Linear(2, 16), nn.ReLU(),
    nn.Linear(16, 16), nn.ReLU(),
    nn.Linear(16, 1)
)
loss_fn = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

This has the same layer widths as the earlier classifier. It is not an identical numerical run: initialization and optimizer details differ between libraries.

The three stages in code

`model.train()` selects training behavior; it does not run fitting by itself. PyTorch normally accumulates gradients, so `zero_grad()` clears the previous ones for our one-update-per-batch procedure. Then `backward()` computes the new gradients, and `step()` changes the parameters. Training-loop reference.

```
model.train()
for epoch in range(100):
    for X_batch, y_batch in loader:
        optimizer.zero_grad()
        logits = model(X_batch)            # forward
        loss = loss_fn(logits, y_batch)
        loss.backward()                    # gradients
        optimizer.step()                   # parameter update
```

The outer loop makes 100 passes through the training set. The inner loop takes the eight batches in each pass, including the final smaller batch. This example therefore makes 800 optimizer updates.

Use the learned model

`model.eval()` selects evaluation behavior. It does not by itself disable gradient recording. `torch.inference_mode()` disables that recording and related overhead. The distinction matters for more elaborate models; this simple network uses only weighted sums, biases, and ReLU hidden activations. Inference-mode reference.

```
model.eval()
X_new = torch.tensor(X_valid, dtype=torch.float32)
with torch.inference_mode():
    logits = model(X_new)
    probabilities = torch.sigmoid(logits)
    predictions = (probabilities >= 0.5)
```

This time sigmoid turns logits into probabilities for interpretation and thresholding. There is no backward call and no optimizer step. Predicting all 120 validation rows leaves the learned parameters unchanged.

Check your understanding

1. What distinguishes a weight, a bias, an activation function, and a hidden-layer width? Why can activation values change during prediction while these stay fixed?
2. For the tiny network, why does a derivative of minus 0.103184 increase a weight under gradient descent?
3. What has changed immediately after backpropagation? What changes after the optimizer step?
4. Why can a large learning rate increase the loss even when the gradient gives a downhill direction locally?
5. With 240 training rows and batch size 32, how many updates occur in an epoch? What changes if the batch size is 240?
6. Why do eight predictions made together not count as eight training updates?
7. Why can a checkpoint have lower log loss but lower classification accuracy than another?
8. What does 100% training accuracy mean in the random-label example? What evidence tests generalization?
9. Which comparison in the noisy-regression experiment demonstrates overfitting? Why is "large versus small" insufficient?
10. Why are logits passed directly to the PyTorch loss, but converted with sigmoid when making probability predictions?

Notebook, review, and sources

Open the companion notebook in Colab to run the experiments. Before changing a setting, predict what will happen to training and validation performance. The Meeting 2 review quiz and in-class notes upload are in the class module.

These notes accompany the Meeting 22 interactive slides and notebook. Their explanations have been revised using the Meeting 2 recording transcript; the PyTorch section is a follow-up, not a claim about material reached in the recording. All datasets are synthetic. Plots use the supplied experiment results, generated with seed 22, scikit-learn 1.9.1, and NumPy 2.5.4. The tiny-network values are checked against direct computation; the landscape is explicitly illustrative. Different library versions or platforms may produce small numerical differences.

1. scikit-learn, Neural network models and MLPClassifier settings.
2. PyTorch, Linear layers, binary cross-entropy with logits, and the optimization loop.